Uppsala University



Machine Learning

Assignment 3

General information

- The recommended tool in this course is R (with the IDE R-Studio). You can download R **here** and R-Studio **here**. You can use Python and Jupyter Notebooks, although the assignments may use data available only through the R package, a problem you would need to solve yourself.
- Report all results in a single, `*.pdf`-file. *Other formats, such as Word, Rmd, Jupyter Notebook, or similar, will automatically be failed.* Although, you are allowed first to knit your document to Word or HTML and then print the assignment as a PDF from Word or HTML if you find it difficult to get TeX to work.
- The report should be written in English.
- If a question is unclear in the assignment. Write down how you interpret the question and formulate your answer.
- You should submit the report to **Studium**. Deadlines for all assignments are **Sunday 23.59**. See **Studium** for dates. Assignments will be graded within 10 working days from the assignment deadline.
- To pass the assignments, *you should answer all questions not marked with **, and get at least 75% correct.
- To get VG on the assignment, *also the questions marked with a ** should be answered. Sometimes you can choose between different VG assignments, then this is explicitly stated. To get VG you need 75% both on questions to pass and the VG part of the assignment. VG will only be awarded on the first deadline of the assignment.
- A report that does not contain the general information (see the **template**), will be automatically rejected.
- Every assignment report must include one oral exam question of your own together with a short model answer, following the oral exam question contribution instructions in this assignment. You are not allowed to use large language models, such as ChatGPT, at all for this contribution.
- State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.
- When working with R, we recommend writing the reports using R markdown and the provided **R markdown template**. The template includes the formatting instructions and how to include code and figures.
- Instead of R markdown, you can use other software to make the pdf report, but you should use the same instructions for formatting. These instructions are also available in **the PDF produced from the R markdown template**.
- The course has its own R package `uuml` with data and functionality to simplify coding. To install the package just run the following:

```
1. install.packages("remotes")
```

2. `remotes::install_github("MansMeg/IntroML", subdir = "rpackage")`

- We collect common questions regarding installation and technical problems in a course Frequently Asked Questions (FAQ). This can be found **here**.
- You are not allowed to show your assignments (text or code) to anyone. Only discuss the assignments with your fellow students. The student that show their assignment to anyone else could also be considered to cheat. During scheduled support sessions held on Zoom, only share your screen in a separate room with a teaching assistant.
- You may use large language models (LLMs), such as ChatGPT, like you may discuss with fellow students: to learn, ask for explanations, and get feedback. You may not use LLMs to produce the solution you submit. You must be able to explain every part of your own assignment at the oral exam, including your code, results, and reasoning, even when it is not covered by the published oral exam questions. If course staff have concerns about your understanding or the authorship of any part of your submission, you may be asked to explain that part at the oral exam.
- Where an exercise explicitly asks you to run an LLM and report its output, you may do so for that experiment. Your explanations and analysis must be your own. This permission does not apply to the oral exam question contribution, for which LLM use is prohibited entirely.
- Do not copy or upload the full assignment text into an LLM.
- Scheduled support sessions are for questions about the assignments and course material. Do not hesitate to ask! Ask questions early if instructions are unclear or you encounter possible bugs, so you do not lose time unnecessarily.
- If you have any suggestions or improvements to the course material, please post in the course chat feedback channel, create an issue **here**, or submit a pull request to the public repository.

Contents

Automatic feedback on your R functions 4

Oral exam question contribution 5

1 Feed-Forward Neural Network using Keras and TensorFlow 6

2 * A simple neural network from scratch 9

Automatic feedback on your R functions

The R package `markmyassignment` (MMA) provides automatic feedback on the functions listed in the selected check configuration. Define or load your own functions in R before running the checks. Use the `tasks` argument to check one function by name; omitting it checks every function in the selected configuration. Functions that depend on other functions need those definitions loaded too.

To select the checks for this assignment, run the following code in R:

```
install.packages("markmyassignment") # Install once
library(markmyassignment)
assignment_path <-
  paste("https://github.com/MansMeg/IntroML/",
        "blob/master/assignments/tests/assignment3/assignment3-vg.yml", sep="")
set_assignment(assignment_path)
# List the function names available in this configuration
show_tasks()
# To check one function, replace function_name with its name:
# mark_my_assignment(tasks = "function_name")
# To check your code/functions, just run
mark_my_assignment()
```

The checks provide feedback on selected numerical cases; they do not assess the whole assignment or award marks. Print your complete function definitions in the PDF report as required by the general instructions, even when the checks pass. You do not need to include MMA results in the report. The supplied checks run R code; Python implementations are assessed from the report and cannot be run directly through MMA.

These checks cover the two functions in the from-scratch neural network VG section. The Keras model-building tasks are assessed from your report.

Oral exam question contribution

Write one possible oral exam question of your own for this assignment together with a short model answer. Include both in your assignment report.

The question should:

- concern material from this assignment or the associated reading or lectures,
- be suitable for a 2–4 minute oral answer, possibly using a whiteboard,
- test conceptual understanding rather than only calculation or code,
- not already be included in the oral exam question list and not be very close to an existing question (e.g. just different numbers).

Note! You are not allowed to use large language models, such as ChatGPT, at all for this oral exam question contribution.

State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.

Keras, TensorFlow, and Google Colab

This assignment uses `keras3` and `tensorflow` for R. Use the `keras3` package for all Keras code. Adam is the default optimizer: use `optimizer_adam(learning_rate = 0.001)` unless the task or supplied example specifies another learning rate or explicitly compares optimizers. To work in Google Colab, open the course R setup notebook, save a copy in Google Drive, and run its setup and training-check cells. Wait for "PASS: training and prediction work" before starting the assignment.

Use the assignment report template linked in the notebook to structure your report. Write explanations in text cells and put R code in code cells. Save your notebook and results in Drive, and run the setup cell again at the start of each R session.

For local installation, follow the course R setup guide. Details on the architecture components are available in the `keras3` reference.

1 Feed-Forward Neural Network using Keras and TensorFlow

After completing the course setup, load the MNIST handwritten-digit dataset using `keras3` with the TensorFlow backend. Scale the pixel values from $[0, 255]$ to $[0, 1]$ once. Keep the images as arrays of shape $(n, 28, 28)$; the network's `layer_flatten()` layer will flatten each image.

```
library(keras3)
use_backend("tensorflow")
mnist <- dataset_mnist()
mnist$train$x <- mnist$train$x/255
mnist$test$x <- mnist$test$x/255
```

Data and metrics. Use validation data for model selection and error analysis; reserve the MNIST test set for Task 1.8. Keras holds out the last 30% of training observations before shuffling when using `validation_split = 0.3`; see the tutorial. Loss means sparse categorical cross-entropy; classification error is 1 − accuracy.

1. (1p) As a first step, we visualize a couple of digits. Include the first eight you can find in the training dataset in the report as an image.

   ```
   idx <- 3
   im <- mnist$train$x[idx,,]
   # For upright digits, reverse MNIST's top-to-bottom rows to
   # match image()'s upward y-axis, then transpose (columns=x, rows=y).
   im <- t(apply(im, 2, rev))
   image(1:28, 1:28, im, col=gray((0:255)/255), xlab = "", ylab = "",
           xaxt='n', yaxt='n', main=paste(mnist$train$y[idx]))
   ```

2. (1p) How large is the training and test set?

3. (2p) Implement a feed-forward network in R using `keras3`; see the Keras 3 feed-forward tutorial. Use one hidden layer with 16 sigmoid units and a softmax output layer. Include the model description in your report.

   Call `set_random_seed(123)` before constructing the model. Train with SGD, learning rate 0.05, no momentum, 3 epochs, batch size 128, and 30% validation data. Keep the epoch count, batch size, and validation split fixed for the next task.

   (a) How many parameters do your model have in total?

   (b) How many parameters does the input layer have?

   (c) How many parameters does the output layer have?

   (d) Report the final epoch's validation accuracy and validation loss. Include the training and validation accuracy and loss curves produced by `plot(history)`.

   Hint! With these settings, the final validation accuracy should be roughly 0.84. Small differences between runs and software versions are expected.

4. (2p) Make the following cumulative changes. For each variant, reset the seed to 123 before constructing and compiling a new model with fresh weights. Keep 3 epochs, batch size 128, and the same validation split. Report the baseline and all six variants in one comparison table with final-epoch validation accuracy and loss. Include learning curves for the baseline and final variant (reuse Task 1.3(d)), plus the final model description. Give one brief interpretation of each change, including any decrease in performance.

   (a) Increase the number of hidden units to 128.

   (b) Change the activation function to reLU.

   (c) Change the optimizer to Adam (as in Assignment 1), using learning rate 0.001.

   (d) Add a second layer with 128 hidden units.

   (e) Add dropout with 0.2 dropout probability for both of your hidden layers.

   (f) Add batch normalization for both of your hidden layers.

5. (2p) Conceptually describe how you would use early stopping as regularization. What would you monitor (e.g. `val_loss`), in which direction should it improve, and would you retain the final epoch's weights or restore those from the best epoch? Explain your choices.

   Note! No implementation or additional training is required for this task.

6. (2p) Start from the best-performing configuration in Task 1.4 and try *one to three additional feed-forward configurations*. Briefly motivate each choice. Train each candidate from fresh weights for *at most 20 epochs*, using the same validation split; early stopping is optional. Add their validation accuracy and loss to the comparison table from Task 1.4. Describe your selected architecture and include its final `keras3` code and training and validation curves. Explain your choice, including if none of the candidates improves on Task 1.4.

   Hint! The general idea is to find the optimal capacity of the model. Try to start with a network that overfits the data and then regularize that model to improve the generalization error on the validation set.

Optional early stopping for Task 1.6. This Keras 3 callback monitors validation loss (lower is better) and stops when validation loss no longer improves:

```
early_stop <- callback_early_stopping(
  monitor = "val_loss", mode = "min",
  restore_best_weights = TRUE
)
# In fit(...), add: callbacks = list(early_stop)
```

With `restore_best_weights = TRUE`, the model retains weights from the epoch with the lowest validation loss; `FALSE` (the default) keeps the final epoch's weights. Evaluate the retained model on the validation data to report its accuracy and loss: the final history entry may refer to different weights. The 20-epoch limit still applies.

7. (2p) Identify two digits from the *validation set* that your selected network has classified incorrectly. Visualize them and include the true digit and the predicted digit. Why do you think the network made each classification mistake?

8. (1p) After model selection, evaluate your selected model once on the MNIST test set. Report test accuracy and loss, compare with validation metrics for the same model, and discuss possible reasons for differences. Is test accuracy necessarily lower than validation accuracy? Explain.

2 * A simple neural network from scratch

This task is only necessary to get one point toward a *pass with distinction* (VG) grade. Hence, if you do not want to get a *pass with distinction* (VG) point, you can ignore this part of the assignment.

We will implement a small feed-forward network for the exclusive-or (XOR) function. For two binary inputs, the target is 1 when exactly one input is 1, and 0 otherwise. A nonlinear hidden layer allows the network to represent this pattern.

The example is adapted from Goodfellow et al. (2016, Section 6.1). All details needed for this exercise are given below; no reading of that section is required. For background, follow the general reading instructions for Block 3.

1. (1p) Implement a network with two inputs, two hidden units, and one output as a function `mini_net(X, W, c, w, b)`. Its forward computation is

$$H = \mathrm{ReLU}(XW + c), \qquad \widehat{y} = Hw + b,$$

where $\mathrm{ReLU}(a) = \max(0, a)$ is applied separately to every element, and the scalar output bias b is added to every prediction. Use the supplied parameter values to compute predictions; no training step is required. For n observations, use the following shapes, with one observation per row:

Object	Meaning	R shape
`X`	Input observations	$n \times 2$
`W`	Input-to-hidden weights	2×2
`c`	Hidden-layer biases, repeated across rows	$n \times 2$
`w`	Hidden-to-output weights	2×1
`b`	Output bias	Scalar

The hidden-layer bias vector has two components. In this R example, `c` repeats those same two values in every row so that it can be added to the hidden-layer inputs. With four observations, `c` is a 4×2 matrix representing *two shared biases*. Use n rows when changing the number of observations.

Debugging hint! Check `dim()` after each matrix multiplication and bias addition. The hidden-layer matrices should be $n \times 2$, and the predictions should be $n \times 1$.

Here is an example:

```
W <- matrix(1, nrow = 2, ncol = 2)
c <- matrix(c(0, -1), ncol = 2, nrow = 4, byrow = TRUE)
X <- matrix(c(0,0,1,1,0,1,0,1), ncol = 2)
w <- matrix(c(1, -2), ncol = 1)
b <- 0

mini_net(X, W, c, w, b)

##      [,1]
## [1,]    0
## [2,]    1
## [3,]    1
## [4,]    0
```

```
mini_net(X, W*0.9, c, w, b)

##      [,1]
## [1,]  0.0
## [2,]  0.9
## [3,]  0.9
## [4,]  0.2
```

2. (1p) Now test changing the value $W_{1,1}$ to 0. What do you get for the results of the network?

3. (1p) What is the current output function? Is the network output function reasonable? Do you have any other output function that might be better?

4. (1p) Now implement a mean squared error loss function for the network. It should work as follows.

```
y <- c(0,1,1,0)
mini_net_loss(y, X, W, c, w, b)

## [1] 0
```

```
mini_net_loss(y, X, 0.9*W, c, w, b)

## [1] 0.015
```

5. (1p) Again, change the value $W_{1,1}$ to 0. What is the value of the loss function?

References

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.